

Assignment on Concurrency and Parallel Programming

Assignment Overview

Objective: Implement 4 small programs demonstrating different concurrency concepts with proper documentation.

Language: Python

Submission: Separate files for each part with comments and a README

Total Parts: 4 independent implementations

Due Date: August 27, 2026

Part 1: Race Condition and Basic Synchronization (5 marks)

What You'll Learn

- What is a race condition
- How to use locks/synchronized methods
- Why synchronization is important

Task

Create a simple **Counter** program where multiple threads increment a shared counter.

Requirements

1. Create a Counter class with:
 - A value (starting at 0)
 - An `increment()` method
2. Create 2-3 threads that each call `increment()` 1000 times
3. **First run WITHOUT synchronization** - show incorrect result
4. **Second run WITH synchronization** - show correct result
5. Print the final counter value

Starter Code (Python)

```
import threading
```

```
class Counter:
```

```
    def __init__(self):
```

```
        self.value = 0
```

```
        # Add a lock here for synchronized version
```

```
    def increment(self):
```

```
        # TODO: Add locking for synchronized version
```

```
        temp = self.value
```

```
        temp = temp + 1
```

```
        self.value = temp
```

```
# Create threads and run
```

```
counter = Counter()
```

```
# TODO: Create 3 threads that increment 1000 times each
```

```
# Expected final value: 3000
```

Expected Output

WITHOUT LOCK:

Final counter value: 2847 (WRONG!)

WITH LOCK:

Final counter value: 3000 (CORRECT!)

What to Document

- Why the first version gives wrong results
- How the lock fixes the problem
- Screenshot of both outputs

Part 2: Producer-Consumer Problem (5 marks)

What You'll Learn

- Thread communication
- Using queues for thread-safe operations
- Basic producer-consumer pattern

Task

Create a simple bakery simulation where:

- **Bakers** (producers) make bread and put in a basket
- **Customers** (consumers) take bread from the basket

Requirements

1. Use a **queue/list** as the shared basket (max size: 5)
2. Create 1-2 Baker threads that:
 - Make bread items (just numbers or names)
 - Add to basket
 - Sleep briefly between making items
3. Create 1-2 Customer threads that:
 - Take bread from basket
 - "Eat" it (print message)
 - Sleep briefly
4. Use proper synchronization so basket doesn't overflow or underflow
5. Run for 10-15 items then stop gracefully

Starter Code (Python)

```
import threading
```

```
import time
```

```
import queue
```

```
basket = queue.Queue(maxsize=5) # Thread-safe queue
```

```
def baker(name):
```

```
    for i in range(5):
```

```
        bread = f"Bread-{i} by {name}"
```

```
        basket.put(bread) # Automatically waits if full
```

```
        print(f"{name} made {bread}")
```

```
        time.sleep(1)
```

```
def customer(name):
    for i in range(5):
        bread = basket.get() # Automatically waits if empty
        print(f"{name} ate {bread}")
        time.sleep(1.5)

# TODO: Create baker and customer threads
```

Expected Output

```
Baker-1 made Bread-0
Baker-1 made Bread-1
Customer-1 ate Bread-0
Baker-1 made Bread-2
Customer-1 ate Bread-1
Baker-1 made Bread-3
...
```

What to Document

- How the queue prevents overflow/underflow
- What happens if basket is full or empty
- Screenshot of output

Part 3: Simple Dining Philosophers (5 marks)

What You'll Learn

- Deadlock problems
- Resource sharing between threads
- Simple deadlock prevention

Task

Implement a **simplified** Dining Philosophers with 3 philosophers and 3 forks.

Requirements

1. Create 3 Philosopher threads
2. Create 3 Fork objects (can use locks)
3. Each philosopher:
 - Thinks (sleep 1-2 seconds)
 - Tries to pick up left fork
 - Tries to pick up right fork
 - Eats (sleep 1-2 seconds)
 - Puts down both forks
 - Repeats 2-3 times
4. **Prevent deadlock** using one simple method:
 - **Option A:** Number forks and always pick lower number first
 - **Option B:** Last philosopher picks right fork first
5. Print clear messages showing what each philosopher is doing

Starter Code (Python)

```
import threading
import time

forks = [threading.Lock() for _ in range(3)]
```

```
def philosopher(id):
    left_fork = id
    right_fork = (id + 1) % 3

    for _ in range(3): # Eat 3 times
        print(f"Philosopher {id} is thinking...")
        time.sleep(1)

        print(f"Philosopher {id} is hungry")
        # TODO: Pick up forks with deadlock prevention
        # TODO: Eat
        # TODO: Put down forks

# TODO: Create 3 philosopher threads
```

Expected Output

```
Philosopher 0 is thinking...
Philosopher 1 is thinking...
Philosopher 2 is thinking...
Philosopher 0 is hungry
Philosopher 0 picked up fork 0
Philosopher 0 picked up fork 1
Philosopher 0 is eating...
Philosopher 1 is hungry
Philosopher 1 waiting for fork...
Philosopher 0 finished eating
...
```

What to Document

- Which deadlock prevention method you used
- Why it prevents deadlock
- Screenshot of successful execution

Part 4: Thread Pool for Parallel Tasks (5 marks)

What You'll Learn

- Using thread pools
- Parallel execution benefits
- Built-in language features for concurrency

Task

Create a simple program that processes multiple tasks in parallel using a thread pool.

Requirements

1. Create 10 simple tasks (examples):
 - Calculate factorial of a number
 - Count from 1 to 1000
 - Simulate processing (sleep for random time)
2. **First:** Execute all tasks sequentially - measure time
3. **Second:** Execute using a thread pool (3-4 threads) - measure time
4. Compare execution times
5. Print which worker thread processed which task

Starter Code (Python)

```
import concurrent.futures
import time
import random

def process_task(task_id):
    """Simulate some work"""
    print(f"Task {task_id} starting on thread {threading.current_thread().name}")
    time.sleep(random.uniform(0.5, 2)) # Simulate work
    print(f"Task {task_id} completed")
    return task_id

# Sequential execution
start = time.time()
for i in range(10):
    process_task(i)
sequential_time = time.time() - start

# Parallel execution with ThreadPool
start = time.time()
with concurrent.futures.ThreadPoolExecutor(max_workers=4) as executor:
    results = executor.map(process_task, range(10))
parallel_time = time.time() - start

print(f"\nSequential: {sequential_time:.2f} seconds")
print(f"Parallel: {parallel_time:.2f} seconds")
print(f"Speedup: {sequential_time/parallel_time:.2f}x")
```

Expected Output

Sequential Execution:

Task 0 completed

Task 1 completed

...

Time: 15.3 seconds

Parallel Execution:

Task 0 starting on ThreadPoolExecutor-0_0

Task 1 starting on ThreadPoolExecutor-0_1

Task 2 starting on ThreadPoolExecutor-0_2

Task 3 starting on ThreadPoolExecutor-0_3

Task 0 completed

Task 4 starting on ThreadPoolExecutor-0_0

...

Time: 4.2 seconds

Speedup: 3.64x

What to Document

- Why parallel execution is faster
- How many worker threads you used
- Screenshots of both execution times

Submission Guidelines

File Structure

```
YourName_Concurrency/
├── README.md
├── part1_race_condition.py
├── part2_producer_consumer.py
├── part3_dining_philosophers.py
├── part4_thread_pool.py
└── screenshots/
    ├── part1_output.png
    ├── part2_output.png
    ├── part3_output.png
    └── part4_output.png
```

What to Include in Each File

1. **Comments** explaining what the code does
2. **Clear output** showing the program working
3. **Your name** at the top

README.md Should Include

1. **How to run** each program
2. **Brief explanation** of what each program demonstrates (2-3 sentences)
3. **Your observations** about what you learned

README Template

Concurrency Assignment - [Your Name]

How to Run

- Part 1: `python part1\_race\_condition.py`
- Part 2: `python part2\_producer\_consumer.py`
- Part 3: `python part3\_dining\_philosophers.py`
- Part 4: `python part4\_thread\_pool.py`

Part 1: Race Condition

This program demonstrates...

I learned that...

Part 2: Producer-Consumer

This program demonstrates...

I learned that...

[etc.]

Helpful Resources

Python:

- `threading` module documentation
- `queue.Queue` for thread-safe queues
- `concurrent.futures` for thread pools